Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

Jev, RAGAS, TorchTPU, Scion, RoPE and RAG variants

a. Jev

What it is: a **non-generative** model from **TypeSafe AI**, released in September 2026 as its first "System One" model. Instead of writing free text, Jev receives a **structured state** (JSON) plus a **closed question** whose possible answers your application defines *before* inference, and returns a **typed decision** with a probability distribution over those answers.

Primitives: Choice (pick one option), **Score** (ordered levels, e.g. 1–5) and a single **yes / no** condition. Because the output is a probability on a fixed scale, your code decides with a threshold:

$$\text{Jev}(\text{state}, \text{question}) \to P(\text{answer} = a), \quad a \in \{a_1, \ldots, a_K\} \quad \Rightarrow \quad \text{act if } P(\text{yes}) \geq \tau$$

Uses: routing and classification, guardrails, reranking retrieved passages, and checking whether a passage supports a claim (groundedness). The idea: use a cheap, fast, predictable judge for decisions, and keep the generative LLM only for writing. **Caveats:** it is new; vendor notes say it can read instructions literally, lose accuracy with long irrelevant state, and be steered by adversarial content, so validate it on your own data.

b. RAGAS (RAG Assessment)

An open-source framework to **evaluate RAG pipelines**, mostly with LLM-as-a-judge and without needing human labels for every metric. It scores the two halves of RAG separately:

Metric	Evaluates	Question it answers
Faithfulness	Generation	Is every claim in the answer supported by the retrieved context? (anti-hallucination)
Answer relevancy	Generation	Does the answer actually address the question?
Context precision	Retrieval	Are the relevant chunks ranked at the top?
Context recall	Retrieval	Did retrieval bring all the information the reference answer needs?

$$\text{Faithfulness} = \frac{\#\,\text{claims in the answer supported by the context}}{\#\,\text{claims in the answer}} \in [0, 1]$$

Typical use: a test set of (question, contexts, answer, reference), re-scored on every change of chunking, embeddings, top-k or prompt, like CI for RAG.
In code: `ragas.evaluate(dataset, metrics=[faithfulness, answer_relevancy, ...])`.

c. TorchTPU

Google's stack to run **PyTorch natively on TPUs**, presented as the **successor of PyTorch/XLA**. Goal: "it should feel like PyTorch"; in the simplest case you only change the device to `"tpu"` and keep the training loop.

- **Native device:** tensors are ordinary PyTorch tensors on the TPU (via the `PrivateUse1` interface), and the distributed APIs (DDP, FSDPv2, DTensor) work.
- **Eager modes:** *Debug* (one op at a time, for finding errors), *Strict* (asynchronous, like default PyTorch) and *Fused* (automatically fuses ops; Google reports 50–100%+ faster than Strict).
- **Compilation:** `torch.compile` captures the graph with Dynamo and uses **XLA** as the backend compiler (instead of Inductor). Custom kernels can be written in Pallas / JAX.
- **vs PyTorch/XLA:** it supports divergent per-rank code (e.g. logging only on rank 0), which PyTorch/XLA's pure SPMD model did not.

Why it matters: most ML code is PyTorch, and NVIDIA's CUDA ecosystem was the main lock-in; TorchTPU makes TPUs a more direct alternative. Google says the public repository and items like vLLM integration are planned for 2026.

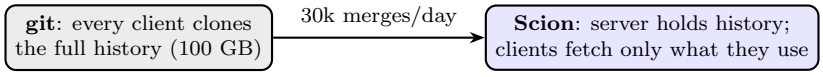
d. Scion (Anthropic's replacement for git)

What it is: the in-house **source control system** Anthropic built to replace git in its monorepo, described in its engineering post of October 2026. Command-line client: `scn`. It is **not** public: Anthropic says it is not a suitable git replacement for general use and shares the design ideas instead.

Why: Claude writing code made merges grow from $\sim$ 2,500 per day (May 2026) to a peak of 30,401 (9 September 2026), with $\sim$ 25 million CI checkouts per day, a history near 100 GB and working copies over 25 GB. Git assumes every client keeps a **full local copy**; most clients were now always-online agents and CI jobs, and shallow / partial clones and mirrors could no longer keep up.

Key design ideas:

- **History lives on the server:** commit, diff, blame and merge run server-side; clients do not download the whole history.
- **Changesets:** each change records only the files modified against its parent; full trees are derived on demand. Metadata in Spanner, file contents in Cloud Storage, deduplicated by hash.
- **Fetch on demand:** clients request just the trees and files they need.
- **Three cache tiers** (per machine, regional, server) answering the same requests; content-addressed objects never change, so caches never need invalidation.
- **Merging:** no globally locked merge queue; testing after merge and before deploy, with Claude fixing conflicts.

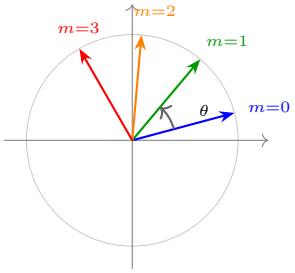


e. RoPE (Rotary Position Embedding)

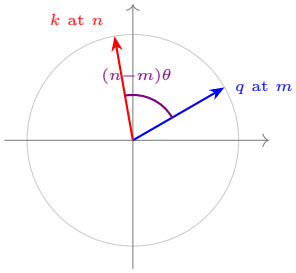
Problem: attention is order-blind; the model needs to know the position of each token. RoPE encodes position by **rotating** the query and key vectors by an angle proportional to the position, instead of adding a position vector.

How: split a d-dimensional vector into $\frac{d}{2}$ pairs. Pair i of the token at position m is rotated by the angle $m\,\theta_i$:

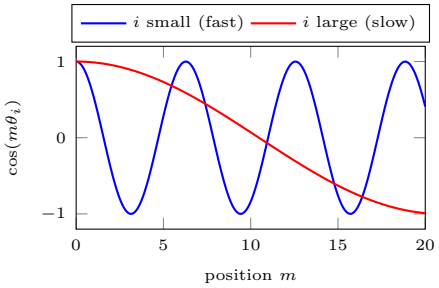
$$\begin{pmatrix} q'_{2i} \\ q'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix} \begin{pmatrix} q_{2i} \\ q_{2i+1} \end{pmatrix}, \qquad \theta_i = 10000^{-2i/d}$$



same word at positions $m = 0..3$: the vector rotates by $m\theta$



the angle between q and k depends only on $n - m$



each pair rotates at its own frequency, like clock hands

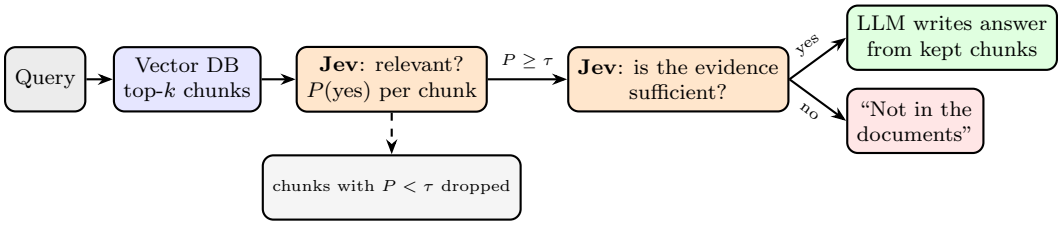
Why it works: rotations preserve length, and the attention score between a query at m and a key at n depends only on their **relative distance**:

$$\langle R_m q,\ R_n k \rangle = \langle q,\ R_{n-m}\, k \rangle$$

Fast-rotating pairs capture nearby order; slow pairs capture long-range position. RoPE adds no parameters, is used by Llama, Qwen, Mistral and most modern LLMs, and allows extending the context window by rescaling θ_i (Position Interpolation, NTK scaling, YaRN).

f. Jev + RAG

Standard RAG retrieves the top-k chunks and sends them all to the LLM, which answers **even when the evidence is weak**, a common cause of hallucinations. **Jev + RAG** keeps the same chunking, embeddings, vector DB and LLM, and puts Jev between retrieval and generation as a **judge**:



(1) **Relevance gate (replaces the reranker):** Jev scores each retrieved chunk with a typed question such as "does this passage help answer the query?"; chunks below the threshold τ are removed. Scores are calibrated probabilities, so chunks from semantic and keyword search can be compared on the same scale.

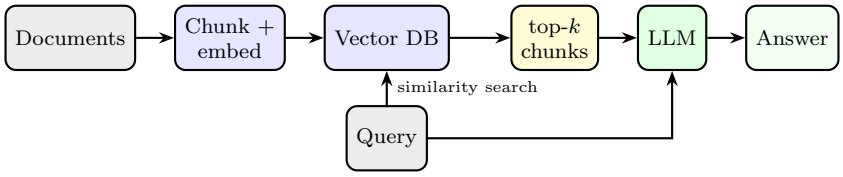
(2) **Sufficiency check:** Jev judges whether the kept evidence is enough. If not, the app **skips the LLM** and answers "not in the documents" instead of guessing.

(3) **Generation:** only clean, sufficient context reaches the LLM.

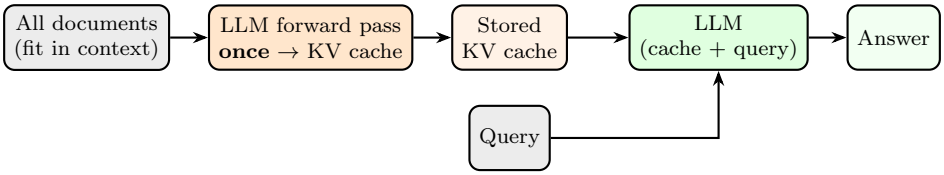
Gains: fewer hallucinations, smaller prompts, an honest "I don't know". **Limit:** Jev cannot recover a chunk that retrieval never found.

g. RAG vs CAG vs KAG

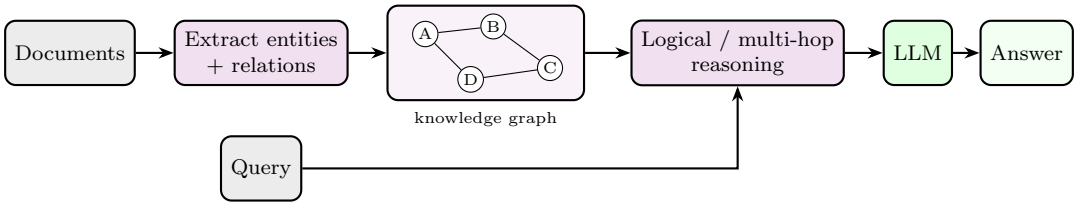
RAG (Retrieval-Augmented Generation): at query time, retrieve the most similar chunks from a vector database and put them in the prompt.



CAG (Cache-Augmented Generation): if the whole knowledge base **fits in the context window**, load it once, precompute its **KV cache**, and reuse that cache for every query. No retrieval step at all.



KAG (Knowledge-Augmented Generation): build a **knowledge graph** (entities and relations) from the documents, and answer by **logical reasoning** over the graph (multi-hop queries) combined with text retrieval. Proposed by Ant Group (OpenSPG) for domains such as finance, law and medicine.



	RAG	**CAG**	**KAG**
Knowledge as	Text chunks + embeddings	Whole corpus in a precomputed KV cache	Knowledge graph + text
At query time	Similarity search, top-k	No retrieval; reuse the cache	Graph reasoning + retrieval
Best for	Large, changing corpora	Small, stable corpora; lowest latency	Multi-hop, factual, rule-heavy domains
Main risk	Missing or irrelevant chunks	Context limit; cost of long context; recompute on updates	Costly graph construction and maintenance